Bauger Cillian

Lien du Repo Github : https://github.com/CillianBgrPro/tasks-api.git

Lundi 2 Novembre

TP1 :

Partie 1 :

Combien de temps à duré le Run ? Ou voit on les lois de chaque étape ?

Mon run à durée 7sec. J'ai accès au lot en cliquant sur Actions, puis sur mon push, ensuite sur hello. Ce qui me liste l'entièreté des logs de chaque étapes.

Partie 3 :

Le test « test_is_valid_title_accepts_normal_title() » à la ligne 13. Grâce à l'affichage des logs on peut déterminer simplement d'où vient l'erreur.

Partie 4:

On attend que la CI soit verte avant de fusionner afin d'éviter de push des bugs sur la branche main, qui doit rester la branche stable et fonctionnelle. Attendre que la CI soit verte permet de détecter les erreurs le plus tôt possible.

Question TP 2 :

Questions : Quelle différence entre un linter (flake8) et un formateur (black) ? Pourquoi séparer lint et test en deux jobs ?

Flake8 detecte des imports inutilisés, des variables non définies, des lignes trop longue et des erreurs de syntaxe. Tandis que le formateur black permet d'imposer un style unique.
Le but de séparer le Lint et le test en deux jobs est d'avoir toutes les infos en un seul run et d'améliorer la visibilité.

Question : Pourquoi tester sur plusieurs version de python ? Que gagne t-on avec la cache ? Un artefact c'est quoi ?

On utilise plusieurs versions de python pour vérifier que le code marche partout où il sera utilisé, car une fonctionnalité ou un comportement peut changer d'une version à l'autre.
Le cache permet de ne pas retélécharger les dépendances à chaque run, donc la CI est plus rapide.
Un artefact est un fichier produit pendant le run (rapport, build…) que GitHub conserve et qu'on peut télécharger après.

Question :

Pourquoi interdire de pousser directement sur la main :

On interdit de pousser directement sur main pour que chaque changement passe par une pull request, la CI vérifie alors le code (lint et tests) avant qu'il soit fusionné, ce qui garantit que main reste toujours stable. Cela permet aussi à quelqu'un de relire les modifications et garde un historique clair de qui a changé quoi et pourquoi.

Question : Pourquoi GIT ne peut il pas résoudre ce conflit seul ? Qu'est ce qui a vérifié le code après la résolution ?

Git ne peut pas résoudre ce conflit seul parce que les deux branches ont modifié le même endroit du fichier : il ne sait pas s'il faut garder une ligne, l'autre ou les deux, c'est donc à un humain de choisir.
Après la résolution, c'est la CI (GitHub Actions) qui a vérifié le code : le push a relancé lint et test sur la PR, et la fusion n'a été possible qu'une fois ces checks au vert.

Questions : Pourquoi GitHub masque-t-il le secret ? Que se passe-t-il si on le commite dans le dépôt ? Secret ou variable : comment choisir ?

GitHub masque le secret pour éviter qu'il apparaisse en clair dans les logs, qui peuvent être lus par d'autres personnes.
Si on le commite dans le dépôt, il reste pour toujours dans l'historique Git et devient visible par tous ceux qui ont accès au code : il faut alors le considérer comme compromis et le remplacer.
On choisit un secret pour une donnée sensible comme un mot de passe, un token ou une clé API, et une variable pour une valeur de configuration non sensible comme le nom de l'environnement ou une URL publique.

Questions : Pour chaque pipeline : quel était le bug, comment l'avez-vous trouvé, comment l'éviter à l'avenir ?

A : mauvaise indentation dans le YAML. Vue en relisant le fichier. À éviter avec un éditeur qui vérifie le YAML.

B : pas de checkout, le code n'était pas récupéré. Vu dans les logs (fichier introuvable). Toujours commencer par checkout.

C : 3.10 lu comme 3.1. Vu dans les logs de setup-python. Toujours mettre les versions entre guillemets.

TP3 : Questions

Questions : Différence entre une image et un conteneur ? À quoi sert le .dockerignore ?

Une image est un modèle figé composé de différentes couches, tandis qu'un conteneur est une instance de cette image en cours d'exécution. Une seul image peut lancer plusieurs conteneurs. Le .dockerfile sert à exclure des fichiers envoyés à docker.

Question : Que gagne-t-on à décrire le lancement dans un fichier plutôt que dans une longue commande ?

Ceci permet que tout le monde lance la meme chose avec une seul commande, sans risque d'oublier une commande. Et permet de gérer facilement plusieurs services ensemble.

Questions : Pourquoi build attend-il test ? Pourquoi éviter le tag latest en production ?

Build attends test afin que l'image soit construite uniquement si les test passent. On évite d'utiliser le tag latest car il ne permet pas de savoir en quelle version il tourne, le déploiement n'est pas reproductible, et il est difficile de revenir précisément a la version précédente.

Question : Quel est l'avantage d'une image versionnée par rapport à « le code de ma machine » ?

L'avantage est quelle permet de faire tourner la meme chose pour tout le monde, les commis sont précis, et une fois publié elle ne change plus.

Questions : À quoi sert un test de fumée ? Pourquoi demander une approbation humaine avant la production ?

Le test e fumée sert a réaliser une vérification rapide juste après le déploiement. Il est nécessaire de demander une approbation humaine avant la production afin de garder une tracée qui a validé quoi, et cela permet également une petite vérification et une protection.

Question : Pourquoi ne pas exécuter une application en root dans un conteneur ?

Car si l'application est compromise, l'attaquant à accès a tout les droits de l'utilisateur qui le fait tourner.

TP4

Questions : Avec vos mots : c'est quoi un pod ? Un deployment ? Pourquoi
un namespace ? (Aidez-vous du vocabulaire ci-dessus et de kubectl get pods -n argocd.)

Un pod c'est un petit conteneur qui tourne et qui peut s'arrêter a tout moment, un deployement c'est « l'instruction » que l'on donne au pod. Un namespace est un espace de rangement dans le cluster.

Question : Pourquoi sépare-t-on le dépôt de l'application et le dépôt de configuration ? (Pensez à ce qui déclenche la CI et à qui a le droit de modifier quoi.)

Pour plus de sécurité, les développeurs modifie task-api tandis que task-config décrit uniquement se qui tourne en production. Cela permet également d'éviter de lancer des tests en boucle sur la pipeline dès que la config est modifiée.

📝 Observation TP4-6 (réponse écrite, pas de capture) : à un moment, avez-vous vu les 5 pods ? Pendant combien de temps environ ? Que s'est-il passé ensuite ? Les deux réponses sont bonnes : si Argo CD a corrigé en quelques secondes, vous pouvez très bien n'en avoir vu que 4, ou jamais 5. Ce qui compte, c'est ce que vous avez vu.

Lorsque j'ai lancé la deuxième commande dans le terminal 2 nouveaux pods sont apparus en pending, il y avait donc bien 5 pods mais moins d'une seconde, et les deux nouveaux n'ont jamais eu le temps de démarrer et sont passés directement en Terminating.

Questions : Que signifie selfHeal ? Pourquoi le retour arrière est-il simple en GitOps ?

SelfHeal sert a ce que si un utilisateur modifie le cluster a la main, ArgoCD le détecte et le remet automatiquement comme dans GIT.
Et le le retour en arrière est simple en GITops car on à juste a faire un git revert.

Questions : Résumez en 5 lignes le chemin d'un git tag jusqu'au pod. Qui pousse (push) et qui tire (pull) ? Quelle différence avec un déploiement CI/CD « classique » ?

Je pousse le tag sur le dépôt talk-api, ensuite Github Actions lance les tests, construit l'image et la publie, Après Staging et mon approbation, le job update-config modifie la version dans deployment.yaml et pousse un commit dans tasks-config. Argo CD, qui surveille tasks-config, détecte que le cluster ne correspond plus à Git. Il applique les changements, et Kubernetes remplace les anciens pods par les nouveaux.
La CI push l'image vers GHCR et le commit vers task-config, tandis que Argo CD pull la configuration depuis GIT, et Kubernetes pull l'image depuis GHCR.
En CI/CD classique c'est la pipeline qui se connecte au cluster pour déployer, hors quand c'est en GitOps, la pipeline ne touche jamais au cluster et c'est ArgoCD qui pull les changements.